ADOPTION GUIDE · INTERNAL · COMPLETE REFERENCE

Tiki.Shared Handbook

Everything in the package as it stands today: the original infrastructure scaffold plus five later additions — gRPC contract packages, universal validation rules, request logging with sensitive-data masking, session-aware HTTP client logging, and tenant-scoped persistence. What got built, what it does at runtime, and how to wire it into a service.

.NET 10 | 14 modules | 5 sibling gRPC contract packages | 138 tests passing | shared-v0.1.0 published

What's in the package

Fourteen modules, one assembly (`Tiki.Shared`). Each ships its own registration extension method — nothing wires itself in silently. Modules with a NEW badge were added after the original scaffold.

Core

Exception hierarchy → `ProblemDetails`, correlation-id + error-handling middleware, `BaseEvent`, universal enums, `PagedResult<T>`, `[Sensitive]`.

no config

Results

`Result<T>` / `Error` — the return type for anything that can fail in an expected way. Throwing is reserved for the unexpected.

no config

Validation

`TikiValidatorBase<T>`, mediator-agnostic `ValidationBehavior`, and `Rules/` — six universal data-shape extension-method groups.

no config

Telemetry

`AddTikiTelemetry()` — OpenTelemetry traces + metrics across HTTP, gRPC, and Kafka spans, exported via OTLP.

Tiki:Telemetry

Caching

`ITieredCache` — L1 memory + L2 Redis, always both. Per-key TTL, skip-L1 option, service-prefixed keys.

Tiki:Caching

Querydsl

Dynamic filter / sort / paginate over any `IQueryable<T>`. Zero EF Core reference. Hard 200-row page cap.

no config

Messaging

`KafkaMessageProducer`, `TikiConsumerBackgroundService<T>`, `RetryDlqRouter` — direct against Redpanda, no framework in between.

Tiki:Messaging,
Tiki:Messaging:RetryDlq

Grpc

Service-token client + server interceptors. Trace context propagates via OTel's gRPC instrumentation, not manual headers.

no config (uses Auth)

Auth

`ServiceContext` — ambient trace id, calling service, **tenant id**, **session id** — plus `IServiceTokenProvider` with an interim HMAC implementation.

Tiki:Auth:HmacServiceToken

HealthChecks

`/health/live` , `/health/ready` — Postgres, Redis, Redpanda, identical shape across every service.

`ConnectionStrings:*`

Logging

`TikiLogEnrichers` , `RequestLoggingMiddleware` , `ClientIpAccessor` , `SensitiveDataMaskingPolicy` — one log line per request plus automatic `[Sensitive]` masking for every sink.

`no config`

Http NEW

`AddTikiExternalHttpClient()` — `SessionLifecycleLoggingHandler` logs started/completed/failed for every outbound call, tagged with session + trace id.

`no config`

Extensions

`TikiJson` shared serializer defaults, `AddTikiCore()` / `UseTikiCore()` baseline wiring.

`no config`

Persistence NEW

`BaseEntity` , a global tenant/soft-delete query filter, and an audit `SaveChangesInterceptor` — Infrastructure-layer only, the one module allowed to reference EF Core.

`no config`

Sibling packages: Grpc.Contracts

Five gRPC contract packages live in this same repo, under `src/Grpc.Contracts/` — but each is its own NuGet package, versioned and released completely independently of `Tiki.Shared` and of each other.

PACKAGE	OWNING SERVICE	STATUS
`Tiki.Grpc.Contracts.Compliance`	Compliance	Fully fleshed — `GetVerificationStatus` RPC
`Tiki.Grpc.Contracts.Identity`	Identity	Placeholder — `Ping` only
`Tiki.Grpc.Contracts.Wallet`	Wallet	Placeholder — `Ping` only
`Tiki.Grpc.Contracts.Transaction`	Transaction	Placeholder — `Ping` only
`Tiki.Grpc.Contracts.Integration`	Integration	Placeholder — `Ping` only

Each references only `Google.Protobuf` and `Grpc.Core.Api` — never `Grpc.Net.Client` or `Grpc.AspNetCore` — so consuming a contract never forces a transport choice. `GrpcServices="Both"` generates both the client stub every caller uses and the service base class the owning service's own implementation derives from, from one `.proto` .

`> terminal — consume a contract`

```
dotnet add package Tiki.Grpc.Contracts.Compliance --version 0.1.0
```

```
services.AddTikiGrpcClient<ComplianceService.ComplianceServiceClient>(
    new Uri("https://compliance-service:5001"));
```

`> terminal — release one contract independently`

```
tools/pack-grpc-contract.sh compliance          # local pack, sanity check
git tag grpc-compliance-v0.1.0                  # must match the csproj <Version>
git push origin grpc-compliance-v0.1.0          # must be reachable from main
```

> terminal — check a contract hasn't silently changed shape

```
tools/hash-grpc-contract.sh compliance
# one SHA-256 over every RPC signature and every message/enum field, sorted
# so declaration order and comments never move it — paste into your OWN
# repo's drift test; this repo does not run that test for you.
```

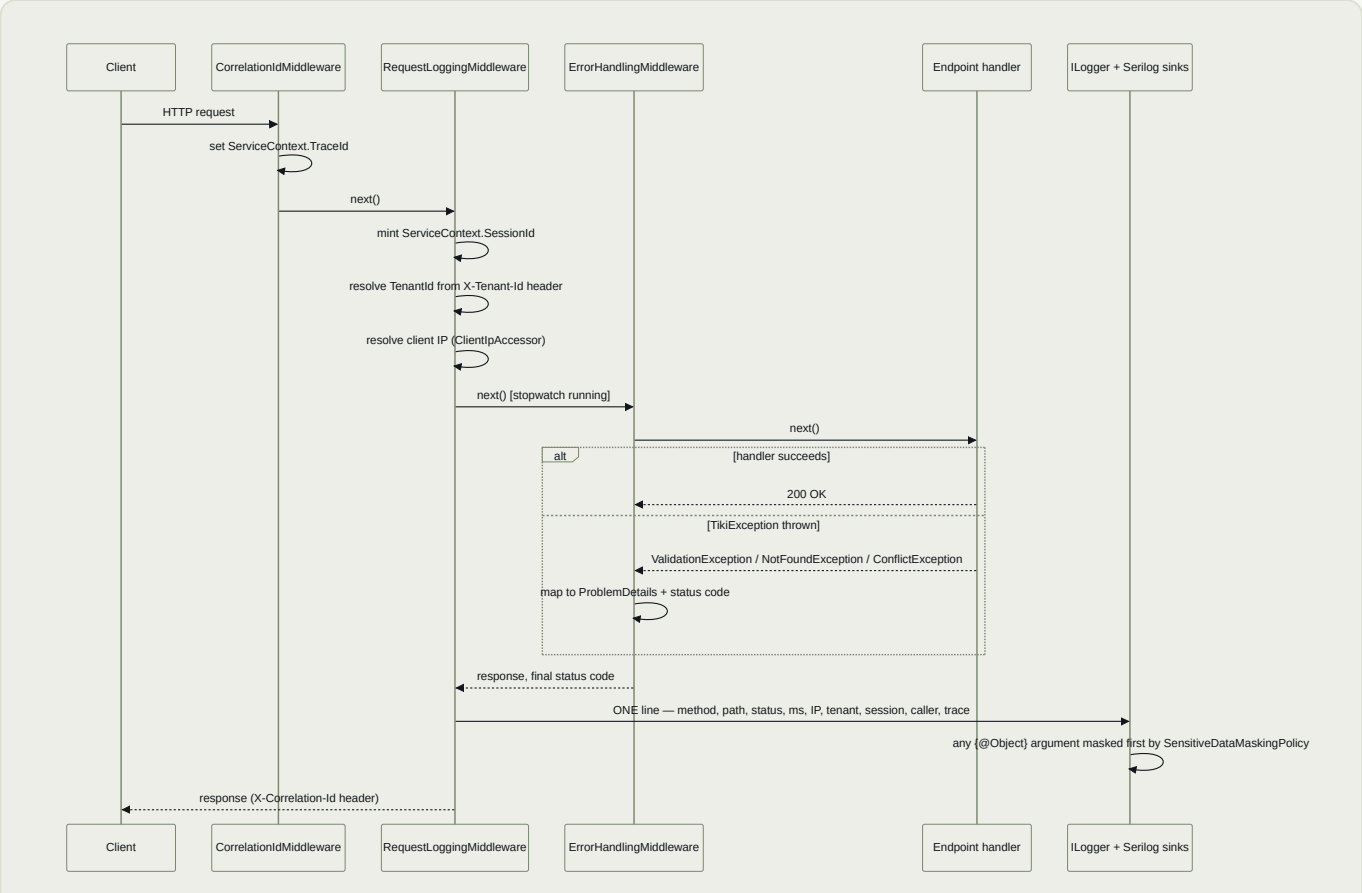
INDEPENDENT RELEASE CADENCE, SAME REPO

`shared-v*.*.*` tags publish `Tiki.Shared` only; `grpc-<service>-v*.*.*` tags publish one named contract only. Both workflows refuse to run unless the tag is reachable from `main` and its version matches the target project's own `<Version>` — never just any project in the repo.

Call flows

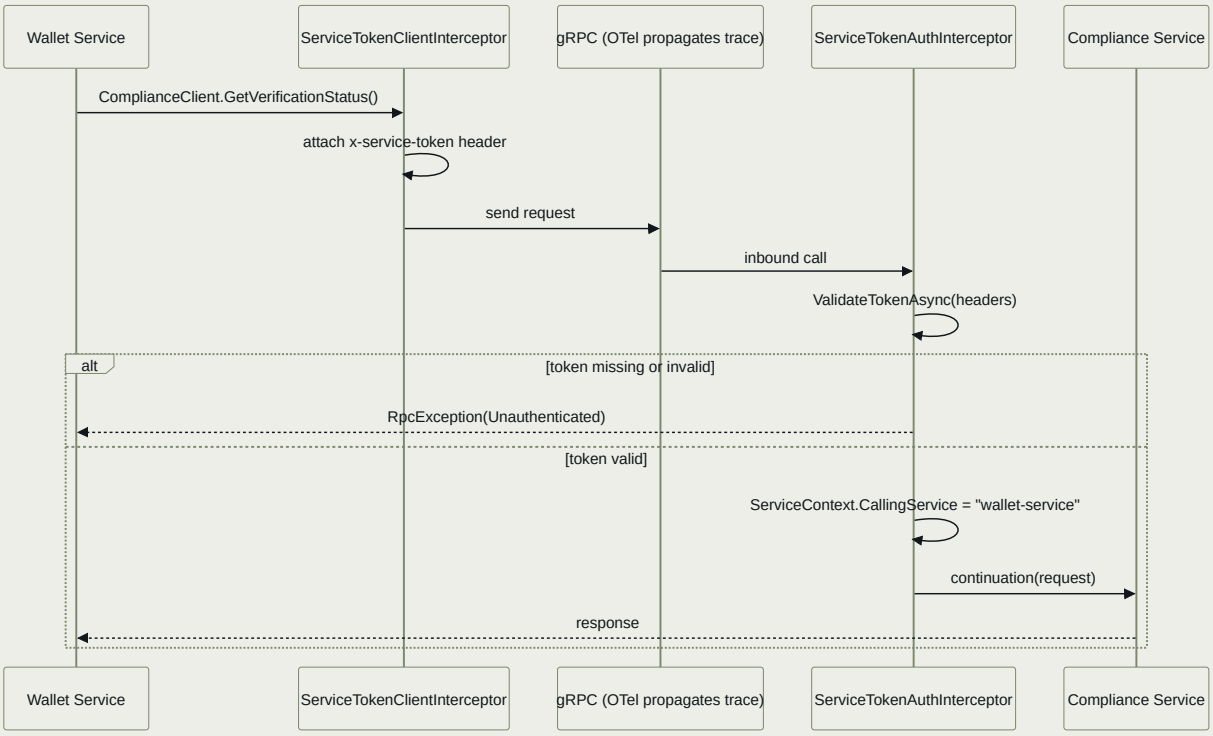
Five shapes a request takes through the package. ● teal is a synchronous call an in-request caller is waiting on; ● violet is Kafka or an outbound HTTP call — fire-and-forget from the pipeline's own point of view.

1. Inbound HTTP request



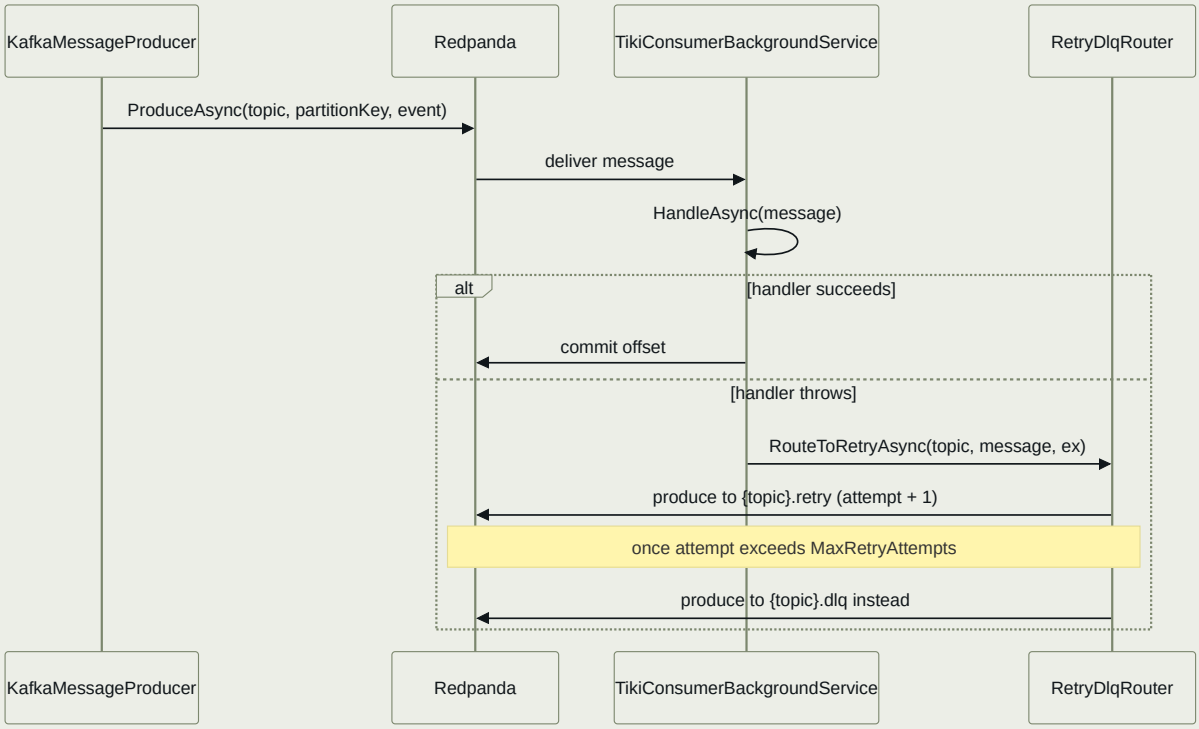
app.UseTikiCore() wires CorrelationId → RequestLogging → ErrorHandling in exactly this order — request logging wraps error handling, not the reverse, so the logged status reflects whatever error handling produced.

2. Outbound gRPC call (sync, in-request)



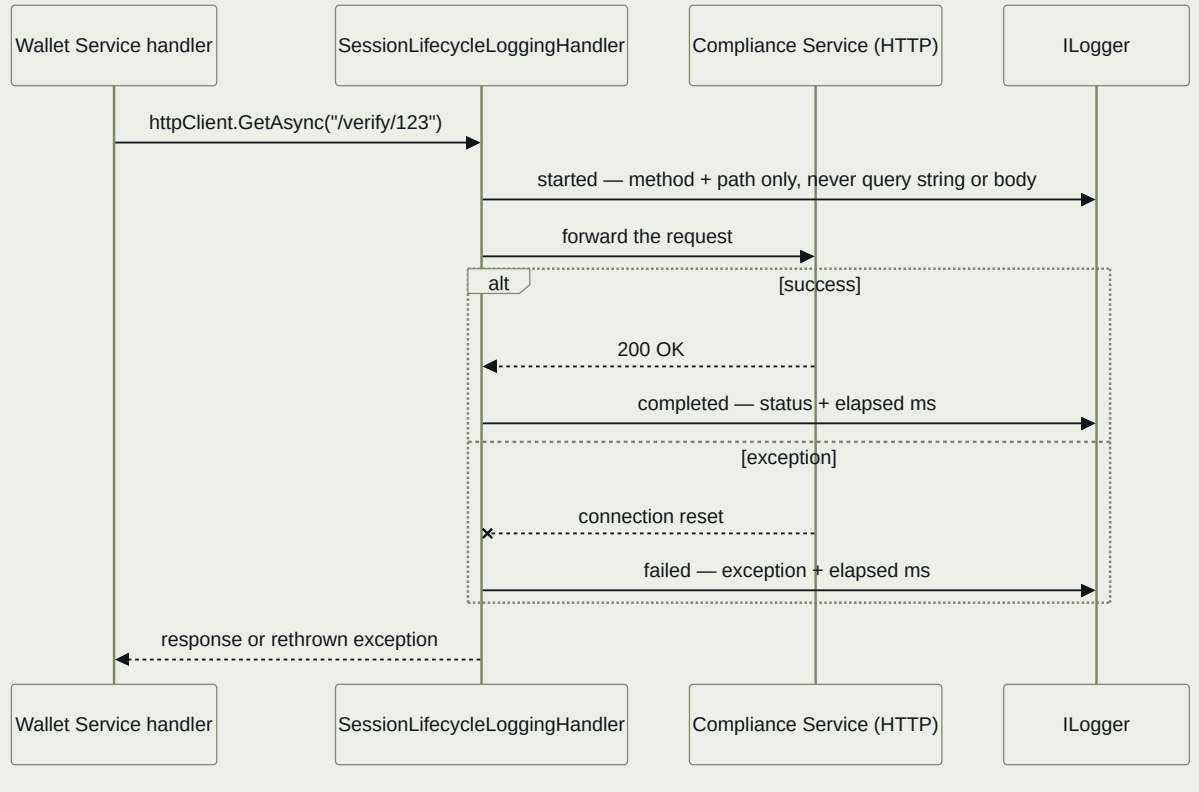
src/Tiki.Shared/Grpc — reserved for calls that need an answer in the same request; everything else goes through Kafka. The contract itself (client stub + service base) ships from a Grpc.Contracts sibling package, not from Tiki.Shared.

3. Kafka publish / consume, with retry → DLQ



`src/Tiki.Shared/Messaging` — delivery is at-least-once, so HandleAsync must be idempotent.

4. Outbound HTTP call, session-lifecycle logged



Every line tagged with the SAME SessionId and TraceId flow 1 logged for the inbound request that triggered this call — filter logs by one SessionId to see the whole downstream tree, in order, timed.

5. Repository read/write through Persistence

```
sequenceDiagram
    participant App as Application handler
    participant Ctx as Your DbContext
    participant Filter as Global query filter
    participant Interceptor as TenantAuditSaveChangesInterceptor
    participant Db as Database

    App->>Ctx: context.Wallets.ToList()
    Ctx->>Filter: TenantId == ServiceContext.TenantId AND NOT IsDeleted
    Filter->>Db: query, filtered
    Db-->>App: only this tenant's, non-deleted rows

    App->>Ctx: context.Wallets.Add(newWallet); context.SaveChanges()
    Ctx->>Interceptor: SavingChanges
    Interceptor->>Interceptor: stamp TenantId, CreatedAt, CreatedBy (Added)
    Interceptor->>Interceptor: stamp UpdatedAt, UpdatedBy (Modified)
    Interceptor->>Db: INSERT / UPDATE, fields already stamped
```

IgnoreQueryFilters() is the sanctioned escape hatch for a genuinely tenant-spanning admin query. Raw SQL (FromSqlRaw) bypasses the filter entirely and needs its own explicit WHERE clause.

Setup, in order

Once the packages are published, this is the whole path from empty service to fully wired-in infrastructure.

1 · Reference the packages

> terminal

```
dotnet add package Tiki.Shared --version 0.1.0
dotnet add package Tiki.Grpc.Contracts.Compliance --version 0.1.0   # only if you call Compliance
```

Pin an exact version — no floating `latest`. A version bump is its own reviewable PR in your service repo, same as any other dependency.

2 · Configure `appsettings.json`

appsettings.json

```
{
  "ConnectionStrings": {
    "Postgres": "Host=localhost;Database=wallet;Username=wallet;Password=***",
    "Redis": "localhost:6379"
  },
  "Tiki": {
    "Telemetry": {
      "OtlpEndpoint": "http://otel-collector:4317",
      "SamplingRatio": 1.0
    },
    "Caching": {
      "DefaultL1Ttl": "00:01:00",
      "DefaultL2Ttl": "00:10:00"
    },
    "Messaging": {
      "BootstrapServers": "redpanda:9092",
      "ConsumerGroupId": "wallet-service",
      "RetryDlq": { "MaxRetryAttempts": 3 }
    },
    "Auth": {
      "HmacServiceToken": {
        "ServiceId": "wallet-service",
        "SharedSecret": "***",
        "TokenLifetime": "00:05:00"
      }
    }
  }
}
```

`TenantId` is not configuration — it arrives per request on the `X-Tenant-Id` header and is resolved by `RequestLoggingMiddleware`.

3 · Wire it up in `Program.cs`

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
const string serviceName = "wallet-service";

builder.Services
    .AddTikiCore()
    .AddTikiTelemetry(serviceName, builder.Configuration)
    .AddTikiCache(serviceName, builder.Configuration)
    .AddTikiMessaging(builder.Configuration)
    .AddTikiHealthChecks(builder.Configuration);

builder.Services.Configure<HmacServiceTokenOptions>(
    builder.Configuration.GetSection(HmacServiceTokenOptions.SectionName));
builder.Services.AddSingleton<IServiceTokenProvider, HmacServiceTokenProvider>();

// only if this service exposes gRPC endpoints of its own
builder.Services.AddGrpc(o => o.Interceptors.Add<ServiceTokenAuthInterceptor>());

// only if this service calls another service over gRPC
builder.Services.AddTikiGrpcClient<ComplianceService.ComplianceServiceClient>(
    new Uri("https://compliance-service:5001"));

// only if this service calls out over plain HTTP
builder.Services.AddTikiExternalHttpClient("compliance");

// only if this service owns a database
builder.Services.AddDbContext<WalletDbContext>(options => options
    .UseNpgsql(builder.Configuration.GetConnectionString("Postgres"))
    .AddInterceptors(new TenantAuditSaveChangesInterceptor(
        () => ServiceContext.TenantId, () => ServiceContext.CallingService)));

Log.Logger = new LoggerConfiguration()
    .ConfigureTikiLogging(serviceName)   // enrichers + [Sensitive] masking, one call
    .WriteTo.Console()
    .CreateLogger();

var app = builder.Build();

app.UseTikiCore();           // correlation id → request logging → error handling
app.MapTikiHealthChecks();   // /health/live, /health/ready
app.MapControllers();
app.Run();
```

THAT'S THE WHOLE BASELINE

Telemetry, caching, messaging, health checks, logging, and the error/correlation pipeline are wired in about a dozen lines. gRPC clients, an external HTTP client, and a `DbContext` are opt-in per feature — only add the ones this service actually needs.

Patterns for consuming services

The shapes every service should reach for, so five repos solve the same problem the same way — worked through in enough depth to copy from directly, not just gestured at.

Application handlers return `Result<T>`, not exceptions

Throw only for the genuinely unexpected. Anything a caller should reasonably expect — not found, a validation failure, a business rule — comes back as data. `Error` carries a machine-readable `Code`, a human `Message`, and an `ErrorType` that `ErrorHandlingMiddleware` and gRPC status mapping both key off.

```
public async Task<Result<WalletDto>> Handle(GetWalletQuery query, CancellationToken ct)
{
    var wallet = await _repository.FindAsync(query.WalletId, ct);
    if (wallet is null)
        return Error.NotFound("wallet.not_found", $"Wallet {query.WalletId} was not found.");

    return wallet.ToDto();
}

// at the call site — Match forces both branches to be handled
var response = result.Match(
    onSuccess: dto => Results.Ok(dto),
    onFailure: error => error.Type switch
    {
        ErrorType.NotFound => Results.NotFound(error),
        ErrorType.Validation => Results.BadRequest(error),
        _ => Results.Problem(error.Message),
    });
```

FACTORY	`ERRORTYPE`	USE FOR
`Error.Validation(code, msg)`	Validation	Malformed or out-of-range input
`Error.NotFound(code, msg)`	NotFound	The entity doesn't exist
`Error.Conflict(code, msg)`	Conflict	State doesn't allow this operation right now
`Error.Unauthorized(code, msg)`	Unauthorized	No valid identity
`Error.Forbidden(code, msg)`	Forbidden	Valid identity, not allowed to do this
`Error.Failure(code, msg)`	Failure	Anything else expected-but-failed

Unexpected failures throw the `TikiException` hierarchy

`ErrorHandlingMiddleware` maps these to `ProblemDetails` automatically — no per-service catch block required.

```
if (!await _lock.TryAcquireAsync(walletId, ct))
    throw new ConflictException($"Wallet {walletId} is locked by another operation.");
```

EXCEPTION	HTTP STATUS	BODY
`ValidationException`	400	`ValidationProblemDetails` — per-property error arrays
`NotFoundException`	404	`ProblemDetails`
`ConflictException`	409	`ProblemDetails`
`TikiException (base)`	400	`ProblemDetails`
`anything else`	500	`ProblemDetails`, detail withheld

Validate request DTOs with the universal rule library

Never a business rule (a transaction limit, an eligibility check) — those stay in the service that owns them. Every method is a `RuleFor(...)` extension, so they compose with any other FluentValidation rule in the same validator.

```
public sealed class CreateTransferRequestValidator : TikiValidatorBase<CreateTransferRequest>
{
    public CreateTransferRequestValidator()
    {
        RuleFor(x => x.Amount).MustBeValidAmount(x => x.CurrencyCode);
        RuleFor(x => x.RecipientPhone).MustBeE164PhoneNumber();
        RuleFor(x => x.RecipientEmail).MustBeValidEmail();
        RuleFor(x => x.Country).MustBeKnownCountry();
        RuleFor(x => x.CurrencyCode).MustBeKnownCurrency();
        RuleFor(x => x.IdempotencyKey).MustBeGuid();
        RuleFor(x => x.ScheduledFor).MustNotBeInTheFuture();
    }
}
```

GROUP	METHOD	CHECKS
`MoneyRules`	`.MustBeValidAmount(x => x.CurrencyCode)`	> 0; decimal places ≤ the currency's ISO 4217 minor units
`PhoneNumberRules`	`.MustBeE164PhoneNumber()`	Leading `+`, no leading zero, 8–15 digits total
`EmailRules`	`.MustBeValidEmail()`	Standard email shape
`CountryCurrencyRules`	`.MustBeKnownCountry()` / `.MustBeKnownCurrency()`	Defined, non-`Unspecified` enum member
`IdentifierRules`	`.MustBeGuid()`	Parses as a GUID
`DateRules`	`.MustNotBeInTheFuture()` / `.MustBeValidDateOfBirth(min, max)` / `.MustBeWithinRange(a, b)`	Temporal bounds

Mark anything that must never be logged in full

Masking happens centrally, in `SensitiveDataMaskingPolicy` — recursively, for any nested object destructured with `@`, not just the top level.

```
public sealed class LoginRequest
{
    public required string Username { get; init; }

    [Sensitive]
    public required string Password { get; init; }

    [Sensitive(SensitiveMaskStrategy.LastFourVisible)]
    public required string CardNumber { get; init; }

    [Sensitive(SensitiveMaskStrategy.Hashed)]
    public required string Ssn { get; init; }

    public required BillingAddress Address { get; init; } // masked recursively if IT has [Sensitive]
properties too
}

logger.Information("Login attempt {@Request}", request);
// Password  → ***REDACTED***
// CardNumber → ************1111
// Ssn        → a stable SHA-256 hex string (same input, same hash, every time)
// Username, Address.* (unless also [Sensitive]) → logged in full
```

Querydsl: building a list endpoint by hand

The full shape a list endpoint accepts and how `QuerydslExecutor` turns it into a filtered, sorted, paged result — without EF Core anywhere in this package, so it works over any `IQueryable<T>` your Infrastructure layer hands it.

The entity being queried

```
public sealed class Transaction
{
    public Guid Id { get; init; }
    public string Reference { get; init; } = string.Empty;
    public decimal Amount { get; init; }
    public TransactionStatus Status { get; init; }        // an enum
    public DateTimeOffset CreatedAt { get; init; }

    [QuerydslIgnore]                    // naming this in a filter/sort is a validation error, not a no-op
    public string InternalRiskNotes { get; init; } = string.Empty;

    [UseRawTextComparison]              // opts this one string property out of case-insensitive matching
    public string ExternalReference { get; init; } = string.Empty;
}
```

Building the request — every piece by hand

```
using Tiki.Shared.Querydsl.Dto;
using Tiki.Shared.Querydsl.Enums;

// "pending or processing transactions over 1000, created this quarter,
//  newest first, page 2 of 20"
var query = new EntityQueryDto
{
    Filters =
    [
        new FilterItem
        {
            PropertyName = nameof(Transaction.Status),
            Operation = FilterOperation.In,
            Value = "Pending,Processing",           // comma-separated for In
            Conjunction = QueryConjunction.And,      // how THIS filter joins the NEXT one
        },
        new FilterItem
        {
            PropertyName = nameof(Transaction.Amount),
            Operation = FilterOperation.GreaterThan,
            Value = "1000",
            Conjunction = QueryConjunction.And,
        },
        new FilterItem
        {
            PropertyName = nameof(Transaction.CreatedAt),
            Operation = FilterOperation.GreaterThanOrEqual,
            Value = "2026-07-01T00:00:00Z",
        },
    ],
    Sorts =
    [
        new SortItem { PropertyName = nameof(Transaction.CreatedAt), Direction = SortDirection.Descending
},
    ],
    Page = 2,
    PageSize = 20,       // requesting 100000 here would be silently capped at MaxPageSize (200)
};
```

Running it

```
var result = QuerydslExecutor.Execute(_dbContext.Transactions.AsNoTracking(), query);

if (!result.IsValid)
{
    // every bad filter/sort is collected, not just the first one —
    // result.Errors is IReadOnlyList<QuerydslFieldError> { PropertyName, Message }
    throw new ValidationException(
        result.Errors
            .GroupBy(e => e.PropertyName)
            .ToDictionary(g => g.Key, g => g.Select(e => e.Message).ToArray()));
}

PagedResult<Transaction> page = result.Value!;
// page.Items, page.TotalCount, page.Page, page.PageSize, page.TotalPages,
// page.HasNextPage, page.HasPreviousPage
```

PROPERTY TYPE	SUPPORTED `FilterOperation`S	NOTES
`string`	Equals, NotEquals, Contains, StartsWith, EndsWith, In, IsNull, IsNotNull	Case-insensitive unless `[UseRawTextComparison]`
`numeric`	Equals, NotEquals, GreaterThan(OrEqual), LessThan(OrEqual), In, IsNull, IsNotNull	`byte` / `short` / `int` / `long` / `float` / `double` / `decimal`
`DateTime / DateTimeOffset`	Equals, NotEquals, GreaterThan(OrEqual), LessThan(OrEqual), IsNull, IsNotNull	No `In`
`Guid`	Equals, NotEquals, In, IsNull, IsNotNull	No ordering — a GUID has none
`enum`	Equals, NotEquals, In, IsNull, IsNotNull	Parsed by member name, case-insensitive
`bool`	Equals, NotEquals	

`IsNull` / `IsNotNull` only apply to a nullable property — asking for them on a non-nullable one is itself a validation error, collected the same way as an unparseable value.

Filter-only count, and the lower-level composable pieces

```
// "how many match, without materializing a page"
var countResult = QuerydslExecutor.Count(_dbContext.Transactions, new EntityCountDto { Filters =
query.Filters });
var totalPending = countResult.Value!.TotalCount;

// or compose the stages yourself, from Tiki.Shared.Querydsl.Extensions
var filtered = _dbContext.Transactions.ApplyFilters(query.Filters);
var sorted   = filtered.ApplySort(query.Sorts);
var page     = sorted.ApplyPaging(query.Page, query.PageSize).ToList();
```

Domain entities inherit `BaseEntity`, get tenancy for free

```
// Domain layer — zero EF Core reference
public sealed class Wallet : BaseEntity
{
    public decimal Balance { get; set; }
    public CurrencyCode CurrencyCode { get; set; }
}

// Infrastructure layer — the only place this touches EF Core
protected override void OnModelCreating(ModelBuilder modelBuilder) =>
    modelBuilder.ApplyTikiConventions(() => ServiceContext.TenantId);
```

Every query against a `BaseEntity`-derived type is automatically scoped to the current tenant and excludes soft-deleted rows — no repository has to remember a `WHERE` clause. Two escape hatches, both used deliberately at the query site:

```
// A genuinely tenant-spanning admin/audit query — visible in review, not buried in a repository
var allTenants = await _dbContext.Wallets.IgnoreQueryFilters().ToListAsync(ct);

// Raw SQL bypasses the filter entirely — write the WHERE clause yourself
var rows = await _dbContext.Wallets
    .FromSqlInterpolated($"SELECT * FROM \"Wallets\" WHERE \"TenantId\" = {tenantId} AND NOT
\"IsDeleted\"")
    .ToListAsync(ct);
```

In-process events, for one service's own internal fan-out

Not for crossing a service boundary — that's `ITikiMessageProducer`, below. `IEventBus` dispatches a `BaseEvent` to every registered handler within the same process, synchronously, no broker involved.

```
public sealed class SendWelcomeEmailOnAccountCreated : IEventHandler<AccountCreatedEvent>
{
    public Task HandleAsync(AccountCreatedEvent @event, CancellationToken ct) =>
_mailer.SendWelcomeAsync(@event.Email, ct);
}

// registration
services.AddScoped<IEventHandler<AccountCreatedEvent>, SendWelcomeEmailOnAccountCreated>();

// dispatch
await eventBus.PublishAsync(new AccountCreatedEvent { TraceId = ServiceContext.TraceId, SourceService =
"identity-service", SubjectId = accountId, Email = email });
```

Multiple handlers for the same event run in `[EventHandlerOrder(n)]` order (lower first); handlers without the attribute run last, in registration order.

Publishing: the partition key is never optional

It must be the id of the entity that needs ordered delivery — `chiRef` for a transaction-scoped event, the sub-account id for a wallet-scoped one.

```
public sealed record BalanceUpdatedEvent : BaseEvent
{
    public required decimal NewBalance { get; init; }
}

await messageProducer.PublishAsync(
    topic: "wallet.balance-updated",
    partitionKey: subAccountId,
    message: new BalanceUpdatedEvent
    {
        TraceId = ServiceContext.TraceId,
        SourceService = "wallet-service",
        SubjectId = subAccountId,
        NewBalance = 1500m,
    });
```

Consuming: every handler must be idempotent

Delivery is at-least-once — a handler failure re-routes through `{topic}.retry`, which means `HandleAsync` can run twice for the same

message. Key your idempotency check off `EventId` or a business key marked `[IdempotencyKey]`.

```
public sealed class BalanceUpdatedConsumer(
    IConsumer<string, string> consumer, RetryDlqRouter router, ILogger<BalanceUpdatedConsumer> logger)
    : TikiConsumerBackgroundService<BalanceUpdatedEvent>(consumer, router, "wallet.balance-updated",
logger)
{
    protected override async Task HandleAsync(BalanceUpdatedEvent message, CancellationToken ct)
    {
        if (await _processedEvents.HasSeenAsync(message.EventId, ct))
            return; // already applied — safe to no-op on redelivery

        await _ledger.ApplyAsync(message, ct);
        await _processedEvents.MarkSeenAsync(message.EventId, ct);
    }
}
```

Topic naming, so five services don't invent five conventions

KIND	CONVENTION	EXAMPLE
Domain event — already happened	`{owning-service}.{event-name}`	wallet.balance-updated
Directed message — a request	`{target-service}.{message-name}`	compliance.screen-transaction-requested
Retry lane	`{topic}.retry`	compliance.screen-transaction-requested.retry
Dead letter	`{topic}.dlq`	compliance.screen-transaction-requested.dlq

gRPC, Kafka, or plain HTTP?

Default to Kafka. Reach for a gRPC client (`AddTikiGrpcClient<T>()`, from a `Grpc.Contracts` sibling package) only when the caller structurally cannot proceed without an answer in the same request. Reach for `AddTikiExternalHttpClient()` for anything outside the Tiki service mesh entirely — a third-party API, for instance. If it can be phrased as "tell the other service what happened," it's a Kafka publish.

Adoption checklist

In order, for the first team wiring a service to the published packages.

01 Confirm the packages are actually published to GitHub Packages before promising a timeline.

02 Add package references at pinned versions; never `latest`.

03 Add the `Tiki` config section to `appsettings.json` (and secrets to your secret store, not source control).

04 Wire `AddTikiCore/Telemetry/Cache/Messaging/HealthChecks`, `ConfigureTikiLogging`, `UseTikiCore()` in `Program.cs`. and

05 Register `IServiceTokenProvider` — `HmacServiceTokenProvider` for now, swapped later with no call-site changes.

06 If this service owns a database: inherit `BaseEntity` for tenant-scoped entities, call `ApplyTikiConventions` in `OnModelCreating`, register `TenantAuditSaveChangesInterceptor`.

07 If this service calls another over gRPC: add the relevant `Tiki.Grpc.Contracts.*` package, wire `AddTikiGrpcClient<T>()`.

08 Confirm `/health/live` and `/health/ready` respond, and that a broker/DB outage flips readiness.

09 Verify one end-to-end trace: an inbound HTTP request, an outbound gRPC call, and a Kafka publish all under one trace id in Jaeger/Tempo.

10 Confirm a request with `[Sensitive]` data in its logged payload never shows the raw value in your log sink.

11 File anything that felt missing or wrong back against this repo — every service should get identical behavior from it.

Judgment calls worth knowing about

Places later additions filled a gap with a reasonable default rather than blocking on a question. Flagged here so any of them can be overridden.

WHERE	THE CALL
Validation	`ValidationBehavior` ships with no hard MediatR dependency (licensing risk) — a structurally-identical interface any mediator can adapt to in one line instead.
Grpc.Contracts	`Tiki.Shared.csproj` had no `<Version>` element; added `0.1.0` so `publish-shared.yml`'s tag-matches-version guard has something to compare against.
Validation/Rules	`IdentifierRules` ships GUID-shape validation only — no existing reference-code format was found in this repo to validate against, so none was invented.
Logging	No ambient tenant concept existed; added `ServiceContext.TenantId`, resolved from an `X-Tenant-Id` header — the source the Persistence module later builds on.
Http	`AddTikiExternalHttpClient` did not exist anywhere in this repo; built the minimal version (session-lifecycle logging only), structured so retry/circuit-breaker handlers can compose onto it later.
Persistence	Deliberately overrides the package's own "no EF Core anywhere" rule for this one module — the base `Microsoft.EntityFrameworkCore` package only, never a provider; `build-test.yml`'s check was updated to match.

Tiki.Shared Handbook · covers the full package as of commits 2810d7d, d1f224a, 3d7c0b1, 8ed276a, 0ccbcd5 on worktree-tiki-shared-five-features · shared-v0.1.0 published to GitHub Packages (verified via publish-shared.yml) · the four placeholder Grpc.Contracts.* packages are not yet independently confirmed published

Syntax error in text

mermaid version 11.17.2